Symbiosis Institute of Technology, Nagpur

PROJECT REPORT

Employee Attrition Prediction using HR Analytics

Industry Domain: **HR Analytics**

Submitted in partial fulfillment of requirement for the award of degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Varun Mundhada

PRN: 23070521184

Under the guidance of

Dr. Priya Dasarwar

Department of Computer Science and Engineering

October 2026

॥वसुधैव कुटुम्बकम्॥

Symbiosis International (Deemed University), Pune

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

DECLARATION

I, hereby declare that the project report titled **"Employee Attrition Prediction using HR Analytics"** submitted herein has been carried out by me at the **Department of Computer Science and Engineering, Symbiosis Institute of Technology, Nagpur** towards partial fulfillment of requirement for the award of Degree of Bachelor of Technology in Computer Science and Engineering. The work is original and has not been submitted earlier as a whole or in part for the award of any degree / diploma at this or any other Institution / University.

I hereby assign all copyright rights in this work, including any revised or expanded derivative works, to Symbiosis Institute of Technology, Nagpur. Any content sourced from references, manuals, datasets or other external materials is duly acknowledged and disclaimed.

Name of Student: **Varun Mundhada**
PRN: **23070521184**
Signature:

Place: Nagpur
Date: October 22, 2026

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CERTIFICATE

This is to certify that the project report titled **"Employee Attrition Prediction using HR Analytics"** (Industry Domain: HR Analytics) has been carried out under my supervision by **Varun Mundhada** (PRN: 23070521184) in partial fulfillment of the requirements for the Bachelor of Technology in Computer Science and Engineering. The work submitted is comprehensive, complete and fit for evaluation.

Dr. Priya Dasarwar
Project Guide

Dr. Sagarkumar S. Badhiye
Head of Department

Dr. Nitin Rakesh
Professor and Director

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, **Dr. Priya Dasarwar**, for her guidance, feedback and encouragement throughout this project. Her suggestions on the problem definition and on how to evaluate the models shaped the final work.

I extend my heartfelt appreciation to our Director, **Dr. Nitin Rakesh**, for his continuous support in providing students with the resources and environment for project work. I would also like to express my deepest gratitude to **Dr. Sagarkumar S. Badhiye**, Head of the Department, Computer Science and Engineering, for his constant support and guidance throughout my academic journey.

I thank the faculty members of the Department of Computer Science and Engineering, whose courses in data structures, database systems, machine learning and big data analytics gave me the foundation for this project. I also acknowledge IBM for publishing the HR Analytics Employee Attrition dataset, and the Apache Spark and Streamlit open-source communities whose tools made this project possible.

I used AI writing tools to help structure and draft parts of this report. The design, code, experiments and results described in it are my own.

Lastly, I would like to acknowledge my family and friends for their unwavering support and encouragement.

Varun Mundhada

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ABSTRACT

Employee attrition, the loss of employees through resignation, is one of the most costly problems an organisation's human resources (HR) function faces. Each departure brings hiring and training costs, lost productivity and lost knowledge, and HR teams usually learn of the risk only when the resignation letter arrives. This project builds a system that predicts which employees are likely to leave, explains why, and gives HR the information early enough to act.

The work uses the IBM HR Analytics Employee Attrition dataset of 1,470 employees and 35 attributes, of whom 237 (16.1%) left the organisation. The data is processed with **Apache Spark**: PySpark DataFrames and Spark SQL are used for cleaning and exploratory analysis, and a **Spark MLlib** pipeline performs feature engineering, categorical encoding, scaling and classification. Four classifiers, Logistic Regression, Decision Tree, Random Forest and Gradient-Boosted Trees, are tuned with 5-fold cross-validation on a stratified training set, with class weights to handle the imbalance between leavers and stayers. The decision threshold is chosen from out-of-fold predictions, the model with the best cross-validated AUC is selected, and the result is evaluated once on an untouched, stratified test set.

The best model, **Logistic Regression**, reached a test ROC-AUC of **0.813**, with a recall of 55.6% and an F1-score of 54.0% for employees who left. The analysis confirms overtime, low income and job level, short tenure, low stock-option level, single marital status and low satisfaction as the strongest signals of attrition. Employees working overtime leave at 30.5% against 10.4% for those who do not. A scalability test on synthetically enlarged data up to about 735,000 records shows that the same Spark pipeline runs unchanged at a much larger scale.

The trained model is deployed in a **Streamlit** web application with an HR dashboard, single-employee prediction with a what-if explanation of the risk drivers and suggested HR actions, batch scoring of CSV files, and a model-performance page. The complete source code is published on GitHub.

Keywords: Employee attrition, HR analytics, Apache Spark, Spark MLlib, machine learning, gradient-boosted trees, class imbalance, Streamlit.

CONTENTS

List of Abbreviations **ix**

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Motivation . . . 1
- 1.3 Problem Statement . . . 2
- 1.4 Objectives . . . 2
- 1.5 Scope of the Project . . . 2
- 1.6 Organisation of the Report . . . 3

2 Literature Review **4**
- 2.1 Employee Turnover and its Causes . . . 4
- 2.2 Machine Learning for Attrition Prediction . . . 4
- 2.3 Handling Class Imbalance . . . 5
- 2.4 Apache Spark and MLlib . . . 5
- 2.5 Algorithms Used . . . 5
- 2.6 Summary and Research Gap . . . 6

3 Dataset and Exploratory Data Analysis **7**
- 3.1 Dataset Description . . . 7
- 3.2 Data Quality and Cleaning . . . 7
- 3.3 Class Balance . . . 8
- 3.4 Exploratory Analysis with Spark SQL . . . 8
- 3.5 Correlation Analysis . . . 11
- 3.6 Key Findings of the Analysis . . . 11

4 Methodology and System Design **13**
- 4.1 Overall Methodology . . . 13
- 4.2 System Architecture . . . 13
- 4.3 Feature Engineering . . . 14
- 4.4 Spark MLlib Pipeline . . . 15

4.5 Data Split 15
4.6 Handling Class Imbalance 16
4.7 Model Training and Hyper-parameter Tuning 16
4.8 Model Selection and Threshold Tuning 16
4.9 Evaluation Metrics 17
4.10 Explaining Individual Predictions 17

5 Implementation **18**
5.1 Tools and Technologies 18
5.2 Project Structure 18
5.3 Module Description 19
5.4 Web Application 20
5.5 Testing 20
5.6 Source Code Repository 20

6 Results and Discussion **22**
6.1 Model Comparison 22
6.2 Effect of Threshold Tuning 24
6.3 Confusion Matrix 25
6.4 Feature Importance 25
6.5 Scalability on Larger Data 26
6.6 Business Insights and Recommendations 28
6.7 Limitations 28

7 Conclusion and Future Scope **29**
7.1 Conclusion 29
7.2 Future Scope 30

A Application Screenshots **33**

B How to Run the Project **38**

C Plagiarism Report **39**

D AI Similarity Report **40**

LIST OF FIGURES

3.1 Class balance of the target variable 8
3.2 Attrition rate by overtime and marital status 9
3.3 Attrition rate by job role 9
3.4 Attrition by income, tenure and age 10
3.5 Attrition by business travel and stock option level 10
3.6 Correlation matrix of key numeric variables (Spark MLlib) 11

4.1 Project methodology 13
4.2 System architecture 14
4.3 Spark MLlib pipeline 15

6.1 Test-set metrics by model 23
6.2 ROC and precision–recall curves on the test set 24
6.3 Confusion matrix of the selected model on the test set 25
6.4 Top 15 features of the selected model 26
6.5 Training time versus data size 27

A.1 HR Dashboard page 33
A.2 Predict an Employee: input form 34
A.3 Predict an Employee: risk gauge, drivers and suggested actions 35
A.4 Batch Prediction page 36
A.5 Model Performance page 37

LIST OF TABLES

2.1 Summary of the literature reviewed 6

3.1 Attributes of the dataset, grouped 7

4.1 Engineered features 14
4.2 Hyper-parameter search grids 16

5.1 Tools and technologies used 18

6.1 Model comparison (test-set metrics at the tuned threshold; best model in bold) 22
6.2 Selected hyper-parameters and training time (grid search with 5-fold CV) 22
6.3 Default (0.5) versus tuned threshold on the test set 24
6.4 Top 10 features of the selected model 26
6.5 Training and scoring time as the data grows (Spark local mode, one laptop) 27
6.6 Findings and recommended HR actions 28

7.1 Objectives versus achievement 29

LIST OF ABBREVIATIONS

Abbreviation	**Full form**
AUC	Area Under the (ROC) Curve
CSV	Comma-Separated Values
CV	Cross-Validation
DT	Decision Tree
EDA	Exploratory Data Analysis
FN / FP	False Negative / False Positive
GBT	Gradient-Boosted Trees
HR	Human Resources
LR	Logistic Regression
MLlib	Apache Spark's Machine Learning Library
PR	Precision–Recall
RDD	Resilient Distributed Dataset
RF	Random Forest
ROC	Receiver Operating Characteristic
SQL	Structured Query Language
TN / TP	True Negative / True Positive

1. INTRODUCTION

1.1 Background

People are the largest cost and the most important asset of most organisations. When an employee leaves, the organisation pays to advertise the role, interview candidates, onboard and train the replacement, and absorb months of reduced productivity while the new person learns the job. Studies of turnover put the full cost of replacing an employee at a large fraction of that employee's annual salary, and higher still for skilled and senior roles [1]. Departures also carry costs that are harder to measure: knowledge leaves with the employee, workloads rise for the colleagues who stay, and repeated exits lower team morale.

HR analytics, also called people analytics, applies data analysis to workforce decisions. Organisations already hold the data needed to understand attrition: demographic details, job role and level, compensation, overtime, travel, tenure, promotion history, performance ratings and satisfaction surveys. Analysed together, these records show patterns that are not visible when each employee is considered on their own. Machine learning takes the next step: it learns those patterns from past leavers and stayers and uses them to estimate the risk for each current employee.

1.2 Motivation

Most organisations react to attrition rather than prevent it. HR usually learns that someone is unhappy only when they resign, and by then the decision is made. A model that ranks employees by their risk of leaving lets HR focus retention effort, such as stay interviews, compensation reviews, workload changes or career discussions, on the people most likely to leave, while there is still time to act.

Two further points shaped this project. First, a risk score alone does not tell HR what to do. The system must explain *why* an employee is at risk, so that the action matches the cause. Second, HR data grows with the organisation and is often combined with attendance, survey and system-log data. The processing should therefore be built on a platform that scales, which is why this project uses **Apache Spark** rather than a single-machine library.

1.3 Problem Statement

Given the HR records of an organisation's employees, predict for each employee whether they are likely to leave the organisation, identify the factors that drive that risk, and present the predictions and insights to HR users through an easy-to-use application, using a scalable big-data processing pipeline.

The problem is a supervised binary classification task with an imbalanced target: only about one employee in six leaves, so a model that simply predicts "stays" for everyone would be 84% accurate and completely useless. The evaluation must therefore focus on how well the model finds the leavers (recall), how many of its warnings are correct (precision), and how well it ranks employees by risk (ROC-AUC).

1.4 Objectives

1. To analyse the IBM HR Analytics dataset with Apache Spark (PySpark and Spark SQL) and identify the factors associated with employee attrition.
2. To build a reusable Spark MLlib pipeline for cleaning, feature engineering, encoding and scaling of HR records.
3. To train, tune and compare four classification algorithms (Logistic Regression, Decision Tree, Random Forest and Gradient-Boosted Trees), handling the class imbalance between leavers and stayers.
4. To select the best model and decision threshold using only the development data, and report an unbiased evaluation on a held-out test set.
5. To explain individual predictions in terms HR can act on, and map each risk driver to a suggested retention action.
6. To deploy the model in a working web application for single and batch predictions, and to show that the pipeline scales to larger data.

1.5 Scope of the Project

The project covers the full path from raw data to a working application: data analysis, model development, evaluation, explanation, deployment and documentation. It uses a public, anonymised dataset, so no real employee data is involved. The application runs on a single machine with Spark in local mode. The same code runs on a Spark cluster without change, but cluster deployment, integration with a live HR information system and scheduled retraining are outside the scope and are listed as future work.

1.6 Organisation of the Report

Chapter 2 reviews published work on attrition prediction and on Apache Spark. Chapter 3 describes the dataset and the exploratory analysis. Chapter 4 explains the methodology and system design. Chapter 5 describes the implementation and the web application. Chapter 6 presents and discusses the results, and Chapter 7 concludes with future scope. Screenshots and instructions for running the project are in the appendices.

2. LITERATURE REVIEW

2.1 Employee Turnover and its Causes

Research on turnover has long separated its causes into job-related factors (pay, workload, role, promotion, relationship with the manager), personal factors (age, marital status, career stage) and organisational factors (culture, environment, policies). Allen, Bryant and Vardaman [1] review the evidence behind common retention practices. They show that turnover is costly, that it can be predicted from measurable factors, and that the right intervention depends on why a particular group of employees leaves. That last finding is the reason this project explains each prediction instead of only producing a score.

2.2 Machine Learning for Attrition Prediction

Saradhi and Palshikar [2] framed employee churn as a classification problem, similar to customer churn, and compared naive Bayes, support vector machines, logistic regression, decision trees and random forests. They found that the support vector machine performed best on their data and that churn models are useful only if they are evaluated on the minority class, not on overall accuracy.

Alao and Adeyemo [3] used decision-tree algorithms (C4.5, C5.0, REPTree and CART) on records from a Nigerian institution and produced readable rules that link salary, length of service and age to attrition. Their work showed the value of interpretable models for HR users.

Punnoose and Ajit [4] compared seven classifiers on HR data from a global retailer and found that Extreme Gradient Boosting (XGBoost) gave the best results and was robust to noisy data, an early sign that boosted trees suit this problem.

Zhao et al. [5] ran a large comparison of ten supervised algorithms on small, medium and large HR datasets, including the IBM dataset used in this project. Gradient-boosted tree ensembles were the most reliable across datasets, while simpler linear models stayed competitive on small data.

Fallucchi et al. [6] analysed the same IBM dataset in detail. They studied the relationship of each attribute with attrition and compared Gaussian naive Bayes, logistic regression, k-nearest neighbours, decision tree, random forest and support vector machine classifiers.

They found overtime, monthly income, total working years and age among the strongest predictors, and reported that naive Bayes gave the best recall.

Yahia, Hlel and Colomo-Palacios [7] looked at attrition prediction from a big-data angle. They argued that people analytics should move "from big data to deep data" by combining many HR data sources, and showed that ensemble models with feature selection improve prediction on the IBM data and on larger datasets.

2.3 Handling Class Imbalance

All of the studies above work with data where leavers are a minority. Common remedies are oversampling the minority class, for example with SMOTE [8], undersampling the majority class, or weighting the classes in the loss function. Class weighting changes no records and works natively with every Spark MLlib classifier used here, so this project weights each leaver by the ratio of stayers to leavers.

2.4 Apache Spark and MLlib

Apache Spark [9] is a unified engine for large-scale data processing. It keeps data in memory across a cluster and offers one API for SQL, streaming, graph processing and machine learning, which made it the standard successor to Hadoop MapReduce. Spark MLlib [10] is its machine-learning library. It provides distributed implementations of common algorithms and, through its `Pipeline` API, a way to chain feature transformers and an estimator into one object that can be tuned, saved and reused. Building the attrition model as an MLlib pipeline means the same code can train on the 1,470-record dataset on a laptop or on millions of records on a cluster.

2.5 Algorithms Used

Logistic Regression models the log-odds of leaving as a weighted sum of the features. It is fast, its coefficients are readable, and with elastic-net regularisation it guards against overfitting. **Decision Trees** split the data on one feature at a time and produce readable rules, but a single tree overfits easily. **Random Forests** [11] average many trees, each grown on a bootstrap sample with a random subset of features, which lowers variance. **Gradient-Boosted Trees** [12] build shallow trees one after another, each correcting the errors of the ensemble so far, and are among the strongest methods for tabular data.

2.6 Summary and Research Gap

Table 2.1: Summary of the literature reviewed

Study	**Approach**	**Key finding**
Saradhi & Palshikar (2011)	Churn framing; NB, SVM, LR, DT, RF	SVM best; evaluate on the minority class
Alao & Adeyemo (2013)	C4.5, C5.0, REPTree, CART	Readable rules on salary, service, age
Punnoose & Ajit (2016)	Seven classifiers incl. XGBoost	Boosting best and robust to noise
Zhao et al. (2019)	Ten algorithms, three data sizes	Boosted trees most reliable overall
Fallucchi et al. (2020)	IBM data; six classifiers	Overtime, income, experience, age key
Yahia et al. (2021)	Big data to deep data; ensembles	Ensembles with feature selection improve results

The literature agrees on the main predictors and on the strength of tree ensembles, but most studies stop at a model score computed in a notebook on a single machine. Few of them (i) build the solution on a scalable big-data engine, (ii) separate model and threshold selection from the final test, which is needed for an honest estimate on a dataset this small, (iii) explain each employee's prediction in terms of HR actions, or (iv) deliver a working application HR can use. This project addresses all four.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 Dataset Description

The project uses the **IBM HR Analytics Employee Attrition & Performance** dataset [13], created by IBM data scientists to resemble a real organisation's HR records. It has **1,470 employee records** and **35 attributes**, and the target column `Attrition` records whether the employee left (`Yes`) or stayed (`No`). There are no missing values. Table 3.1 groups the attributes.

Table 3.1: Attributes of the dataset, grouped

Group	**Attributes**
Target	Attrition (Yes / No)
Personal	Age, Gender, MaritalStatus, DistanceFromHome, Education (1–5), EducationField
Job	Department, JobRole, JobLevel (1–5), JobInvolvement (1–4), BusinessTravel, OverTime, PerformanceRating (1–4), TrainingTimesLastYear
Compensation	MonthlyIncome, DailyRate, HourlyRate, MonthlyRate, PercentSalaryHike, StockOptionLevel (0–3)
Satisfaction	EnvironmentSatisfaction, JobSatisfaction, RelationshipSatisfaction (1–4), WorkLifeBalance (1–4)
Tenure	TotalWorkingYears, NumCompaniesWorked, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager
Identifiers / constant	EmployeeNumber (ID), EmployeeCount (always 1), Over18 (always Y), StandardHours (always 80)

3.2 Data Quality and Cleaning

The data was loaded into a Spark DataFrame with schema inference. Spark SQL checks found no missing values and no duplicate records. Three columns, `EmployeeCount`, `Over18` and `StandardHours`, hold the same value for every employee and carry no information, so

they were removed. `EmployeeNumber` is kept only as a record identifier for the data split and is not used as a feature. The target was converted to a numeric label (1 = left, 0 = stayed), and all numeric columns were cast to double precision for MLlib. This leaves 23 numeric and 7 categorical input features.

3.3 Class Balance

Of the 1,470 employees, **237 (16.1%) left** and 1,233 stayed (Figure 3.1). The imbalance is moderate but important: it is why the project uses class weights, a stratified split, a tuned threshold and AUC / recall / F1 as metrics rather than plain accuracy.

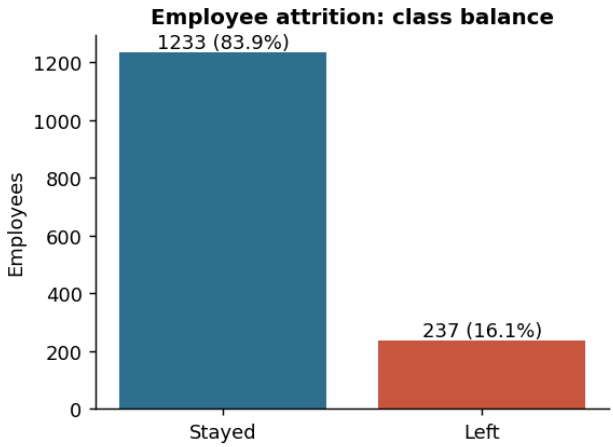
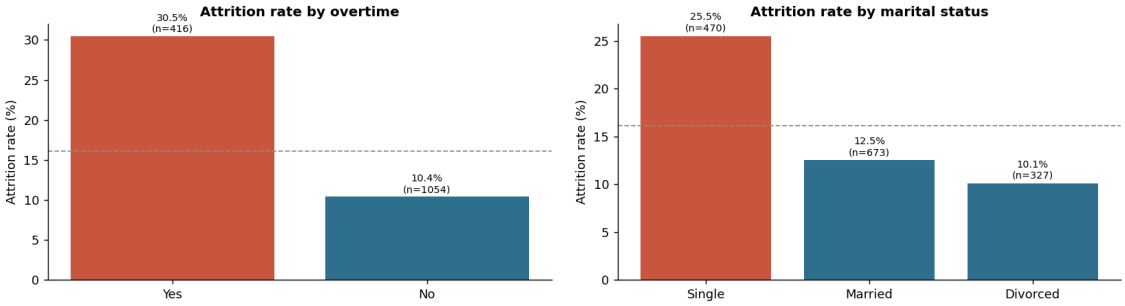


Figure 3.1: Class balance of the target variable

3.4 Exploratory Analysis with Spark SQL

Each factor was analysed with Spark SQL aggregations of the form `SELECT factor, COUNT(*), AVG(label) FROM hr GROUP BY factor`. Continuous variables such as age, income and tenure were first grouped into bands with `CASE WHEN` expressions. In the charts below, bars above the company-wide rate (dashed line) are shown in red.

Overtime is the single strongest categorical signal: employees who work overtime leave at 30.5%, almost three times the 10.4% rate of those who do not (Figure 3.2a).



(a) Overtime (b) Marital status

Figure 3.2: Attrition rate by overtime and marital status

Job role matters a great deal (Figure 3.3). Sales Representatives leave at 39.8%, while Research Directors leave at only 2.5%. Entry-level, lower-paid roles such as laboratory technicians and sales representatives carry the highest risk.

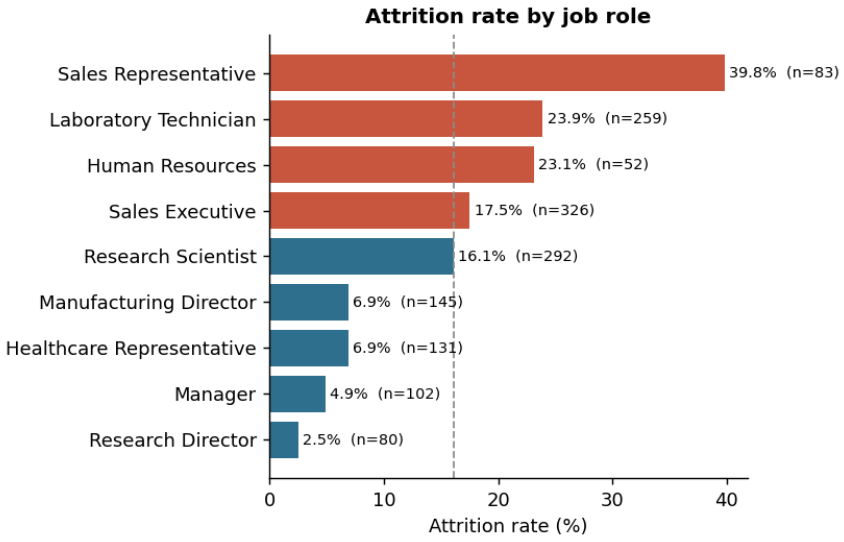
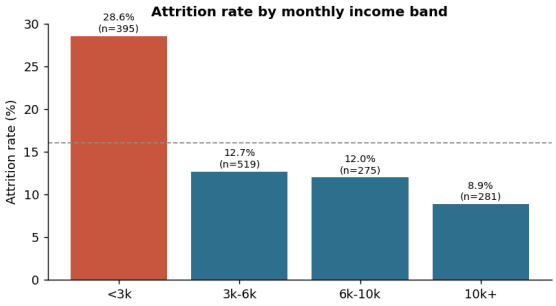
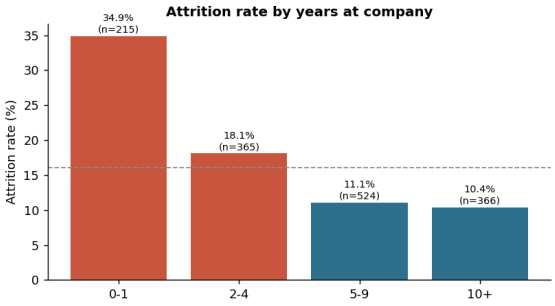


Figure 3.3: Attrition rate by job role

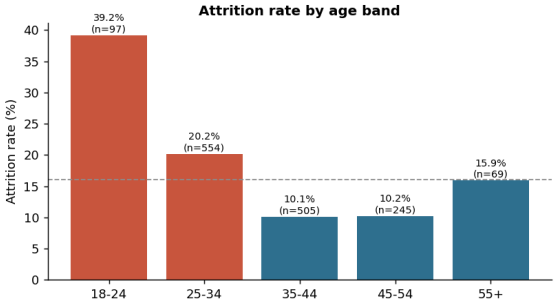
Income, tenure and age (Figure 3.4). Employees earning under 3,000 a month leave at 28.6%, against 8.9% for those above 10,000. Attrition is highest in the first two years at the company (34.9%) and falls steadily with tenure, and employees aged 18–24 leave at 39.2%. These variables are strongly correlated with each other (Figure 3.6), and all describe the same group: young, early-career, lower-paid employees.



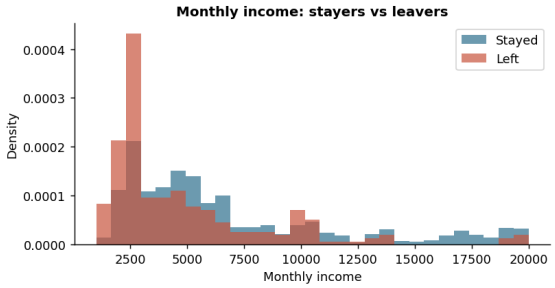
(a) Income band



(b) Years at company



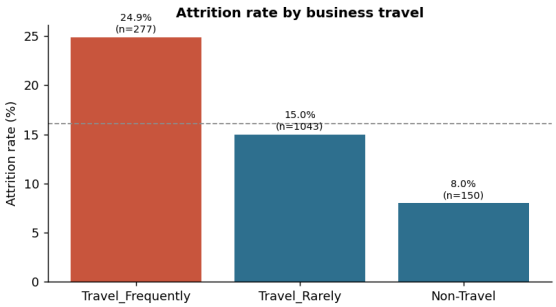
(c) Age band



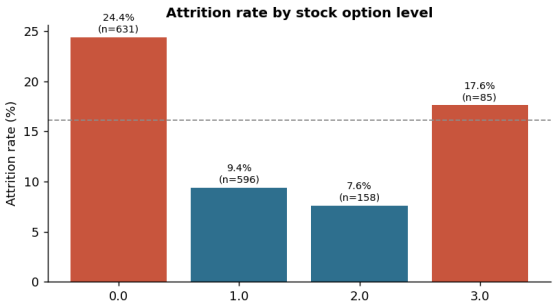
(d) Income distribution

Figure 3.4: Attrition by income, tenure and age

Marital status, travel and stock options (Figures 3.2b and 3.5). Single employees leave at 25.5% against 12.5% for married employees. Frequent travellers leave at 24.9% against 8.0% for those who never travel. Employees with no stock options leave at 24.4%, against 7.6–9.4% at levels 1 and 2, which suggests long-term incentives help retention.



(a) Business travel



(b) Stock option level

Figure 3.5: Attrition by business travel and stock option level

Satisfaction. Employees reporting low job satisfaction leave at 22.8% against 11.3% for very high satisfaction, and those rating work-life balance as "bad" leave at 31.3%.

3.5 Correlation Analysis

The Pearson correlation matrix was computed with Spark MLlib's `Correlation.corr` on an assembled feature vector (Figure 3.6). No single numeric variable correlates strongly with attrition; the largest values are around -0.17, for total working years, job level, monthly income, years in current role and age. Attrition is therefore driven by a combination of factors rather than by any one of them, which is why multivariate models are needed. The matrix also shows strong collinearity: monthly income and job level correlate at 0.95, and the tenure variables at 0.5–0.8. Tree models handle this naturally, and regularisation handles it for logistic regression.

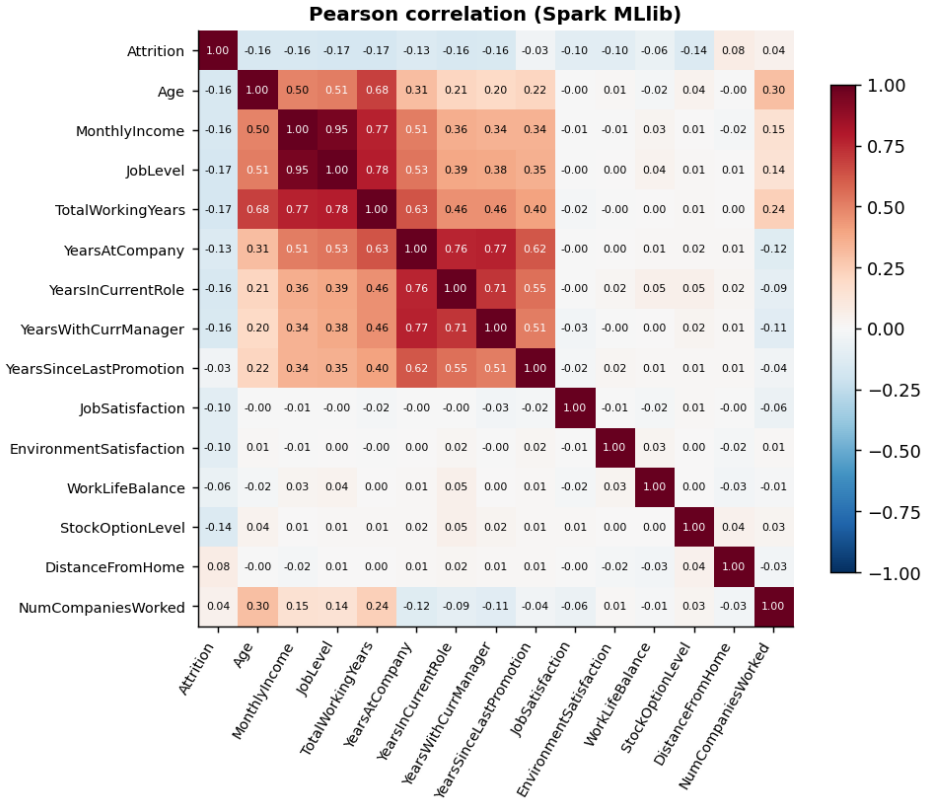


Figure 3.6: Correlation matrix of key numeric variables (Spark MLlib)

3.6 Key Findings of the Analysis

- The highest-risk profile is a young, single employee in an entry-level role (job level 1), in the first two years at the company, working overtime, with low income and no stock options.
- Overtime, job role, tenure, income and stock options separate leavers from stayers most clearly.

- No single variable explains attrition on its own, which justifies a multivariate machine-learning model.
- Several variables measure the same thing (income, level and experience), so regularisation or tree models are preferred.

4. METHODOLOGY AND SYSTEM DESIGN

4.1 Overall Methodology

The project follows the standard data-science life cycle, adapted to Spark (Figure 4.1): understand the business problem, collect the data, explore it with Spark SQL, clean it and engineer features, train and tune models, evaluate and select, deploy, and turn the results into HR actions.



Figure 4.1: Project methodology

4.2 System Architecture

Figure 4.2 shows the architecture. The **data layer** holds the HR dataset and any new records uploaded by users. The **Spark processing layer** loads, cleans and analyses the data with DataFrames and Spark SQL. The **Spark MLlib layer** trains and tunes the models and saves the best complete pipeline. The **model store** keeps the saved model, its metrics and the analysis results. The **Streamlit application** loads the saved pipeline once and serves the dashboard and predictions to HR users.

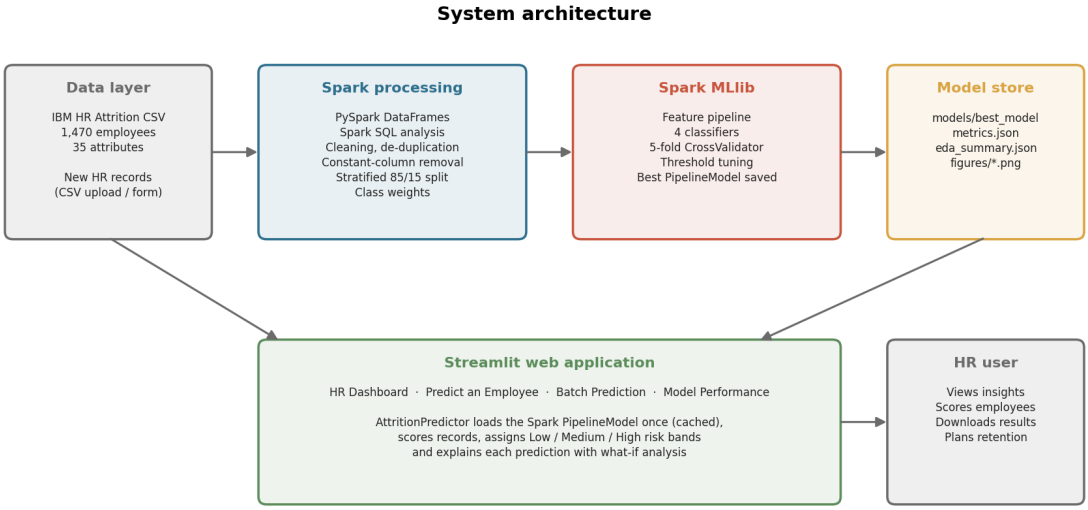


Figure 4.2: System architecture

4.3 Feature Engineering

Five features were derived from the raw attributes to capture relationships the raw columns show only indirectly:

Table 4.1: Engineered features

Feature	**Formula**	**Meaning**
IncomePerJobLevel	MonthlyIncome / JobLevel	Pay relative to level: underpaid for the level
TenureRatio	YearsAtCompany / (TotalWorkingYears + 1)	Share of the career spent at this company
YearsPerCompany	TotalWorkingYears / (NumCompaniesWorked + 1)	Average stay per employer: job-hopping tendency
PromotionGap	YearsSinceLastPromotion / (YearsAtCompany + 1)	How stalled the career is at this company
SatisfactionIndex	Mean of five satisfaction / involvement ratings	One overall engagement score

The features are computed by a `SQLTransformer` stage *inside* the MLlib pipeline. Because of this, the saved model accepts raw HR records directly, and the same formulas are applied identically in training and in the application.

4.4 Spark MLlib Pipeline

Every model shares the same pipeline (Figure 4.3):

1. **SQLTransformer** adds the five engineered features.
2. **StringIndexer** converts the seven categorical columns to numeric indices, with the `keep` option for invalid values, so that an unseen category in new data does not break scoring.
3. **OneHotEncoder** turns each index into a binary vector, so no false order is implied between categories.
4. **VectorAssembler** combines the 23 numeric, 5 engineered and encoded categorical features into one vector.
5. **StandardScaler** centres and scales each feature to mean 0 and standard deviation 1, which logistic regression needs for regularisation to treat features fairly.
6. The **classifier**.

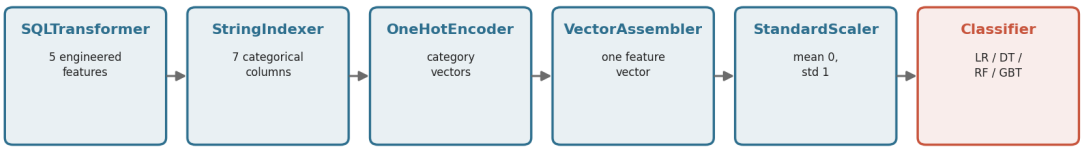


Figure 4.3: Spark MLlib pipeline

4.5 Data Split

With only 237 leavers, the data must be used carefully. It was split into a **development set (85%)** and a **test set (15%)** stratified on the label so that both parts keep the 16.1% attrition rate. Within each class, employees are ranked by a seeded hash of their employee number and the lowest 15% go to the test set. Unlike Spark's random `sampleBy`, whose result depends on how the data happens to be partitioned, this split is exact and identical on every run and every machine, so the reported results can be reproduced. This gave 1249 development employees (201 leavers) and 221 test employees.

- **Development set**: hyper-parameter tuning by 5-fold cross-validation, threshold selection from out-of-fold predictions, and model selection.
- **Test set**: used once, at the end, for the final reported results. It played no part in any choice, so its results are an unbiased estimate.

A three-way split with a separate validation set was tried first. With only about 39 leavers in a 15% validation set, the threshold and model choice changed noticeably with a few employees, and thresholds tuned on it lowered the F1-score on the test set for every model. Cross-validation uses every development record for both fitting and checking, which gives much more stable choices on a dataset of this size.

4.6 Handling Class Imbalance

Each leaver in the development set was given a weight equal to the ratio of stayers to leavers (about 5.2), and each stayer a weight of 1, through MLlib's `weightCol` parameter. The two classes therefore contribute equally to the training loss, and the models do not learn to ignore the minority class.

4.7 Model Training and Hyper-parameter Tuning

Each classifier was wrapped in a `CrossValidator` with a `ParamGridBuilder` grid, 5 folds, ROC-AUC as the selection metric and a parallelism of 4. Because the whole pipeline is cross-validated, encoders and the scaler are re-fitted inside each fold, so no information leaks from the held-out folds.

Table 4.2: Hyper-parameter search grids

Model	**Grid searched**	**Settings**
Logistic Regression	regParam $\in \{0.001, 0.01, 0.1\}$, elasticNetParam $\in \{0, 0.5, 1\}$	9
Decision Tree	maxDepth $\in \{3, 5, 7\}$, minInstancesPerNode $\in \{5, 20\}$	6
Random Forest	numTrees $\in \{100, 200\}$, maxDepth $\in \{5, 8\}$	4
Gradient-Boosted Trees	maxIter $\in \{50, 100\}$, maxDepth $\in \{3, 5\}$ (stepSize 0.1)	4

4.8 Model Selection and Threshold Tuning

A classifier outputs a probability of leaving, and a threshold turns it into a yes / no prediction. The default threshold of 0.5 is rarely the best choice for imbalanced data. For each tuned model, the best parameters were re-fitted five times, each time leaving out one fold of the development set, so that every development employee receives an *out-of-fold* (OOF) probability from a model that did not see them in training. The threshold between 0.10 and 0.90 that maximises the F1-score on these out-of-fold predictions was selected. The

model with the highest mean cross-validated ROC-AUC was chosen as the final model, and `CrossValidator` re-fits it on the whole development set.

4.9 Evaluation Metrics

With TP, FP, TN and FN denoting true / false positives and negatives (positive = left):

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

ROC-AUC is the probability that the model ranks a randomly chosen leaver above a randomly chosen stayer, and it does not depend on the threshold. **Recall** matters most to HR, since a missed leaver is a lost employee. **Precision** matters because every false alarm costs HR time.

4.10 Explaining Individual Predictions

For each prediction the application runs a **what-if analysis**. For each of 19 factors, such as overtime, income, stock options, satisfaction ratings, travel and tenure, it creates a copy of the employee's record in which only that factor is set to its typical value among employees who stayed (the median for numbers, the most common value for categories). All copies are scored in a single Spark job. The drop in the leave probability shows how much that factor contributes to the employee's risk:

$$\text{Impact}(f) = P(\text{leave} \mid x) - P(\text{leave} \mid x \text{ with } f \leftarrow \text{typical}_{\text{stayers}}(f))$$

Factors with a positive impact are listed in order, each with a suggested HR action (for example, "review workload and overtime" or "benchmark pay against the market"). This works with any model, including tree ensembles whose internals are hard to read.

5. IMPLEMENTATION

5.1 Tools and Technologies

Table 5.1: Tools and technologies used

Layer	**Tool**	**Use in the project**
Language	Python 3.11	All code
Big-data engine	Apache Spark 3.5 (PySpark)	DataFrames, Spark SQL, local-mode execution
Machine learning	Spark MLlib	Pipeline, transformers, classifiers, CrossValidator
Runtime	Java 17 (Temurin)	JVM required by Spark
Data handling	pandas, NumPy	Small result sets, metric computation
Visualisation	Matplotlib, Plotly	Report charts; interactive app charts
Web application	Streamlit	HR dashboard and prediction interface
Testing	pytest	Automated tests of the pipeline and predictor
Version control	Git, GitHub	Source control and submission
Documentation	LaTeX (Overleaf)	This report

5.2 Project Structure

```
employee-attrition-prediction/
|-- data/ IBM HR dataset + sample file for batch scoring
|-- src/
| |-- config.py paths, column lists, feature SQL
| |-- spark_session.py local SparkSession (Windows set-up)
| |-- preprocess.py load, clean, split, class weights, pipeline
| |-- eda.py Spark SQL analysis and charts
| |-- train.py training, tuning, selection, evaluation
| |-- predict.py scoring, risk bands, what-if explanation, CLI
| `-- scale_benchmark.py scalability test on enlarged data
```

```
|-- app/streamlit_app.py web application
|-- models/ saved PipelineModel + metrics JSON
|-- reports/figures/ generated charts
|-- tests/ pytest tests
|-- report/ this LaTeX report
`-- ppt/ presentation
```

5.3 Module Description

config.py holds every constant in one place: file paths, the lists of numeric and categorical columns, the SQL for the engineered features and the allowed values for the application's inputs.

preprocess.py loads the CSV into Spark, removes the constant columns, adds the numeric label, performs the reproducible, hash-based stratified development / test split, adds class weights and builds the feature stages of the pipeline.

eda.py registers the cleaned data as a Spark SQL view, computes attrition rates by each factor and band with SQL, computes the correlation matrix with MLlib and saves the charts and a JSON summary.

train.py runs the cross-validated grid search for each of the four models, tunes the threshold on out-of-fold predictions, evaluates on test data, saves the best `PipelineModel` with Spark's model writer and draws the ROC, precision–recall, comparison, confusion-matrix and feature-importance charts.

predict.py contains the `AttritionPredictor` class used by the application. It loads the saved pipeline, converts incoming pandas records to a Spark DataFrame with a fixed schema, returns the probability, a risk band (High at or above the tuned threshold, Medium at or above 60% of it, Low otherwise) and the predicted label, and produces the what-if explanation. It can also be run from the command line to score a CSV file.

The core of the training step for one model is:

```
pipeline = Pipeline(stages=feature_stages() + [classifier])
cv = CrossValidator(estimator=pipeline, estimatorParamMaps=grid,
                    evaluator=BinaryClassificationEvaluator(metricName="areaUnderROC"
                        ),
                    numFolds=5, parallelism=4, seed=42)
best = cv.fit(dev).bestModel # tuned on development data only
threshold = best_threshold(*oof_scores(clf, params, dev)) # out-of-fold
    predictions
test_metrics = metrics_at(*scores(best, test), threshold) # reported once
best.write().overwrite().save("models/best_model")
```

5.4 Web Application

The Streamlit application has five pages, listed in a sidebar that also shows the model in use and its test AUC:

1. **HR Dashboard**: key figures (headcount, leavers, attrition rate, average income) and interactive charts of attrition by overtime, job role, tenure, income, marital status and age. Department, gender and overtime filters update every chart.
2. **Predict an Employee**: a form grouped into Personal, Job, Compensation, Satisfaction and Tenure sections, with rating scales shown in words ("Low", "Very High"). The result shows a gauge of the leave probability against the threshold, the risk band, and a table of the top risk drivers with suggested HR actions. Inputs are checked for consistency, for example that years at the company do not exceed total working years.
3. **Batch Prediction**: upload a CSV of employee records, or use the bundled sample. The file is validated, every employee is scored with Spark, and the page shows the counts by risk band, a risk-band chart, the average risk by department and a ranked table, with the results available for download.
4. **Model Performance**: the comparison of all four models, the ROC curve, the confusion matrix and the feature importance of the selected model.
5. **About**: a summary of the project and the method.

The Spark session and model are loaded once and cached with `st.cache_resource`, so later predictions return in about a second.

5.5 Testing

Automated tests (`pytest`) check that cleaning keeps all 1,470 records and removes the constant columns, that the three-way split is disjoint, covers every record and preserves the attrition rate, that the predictor scores the sample file with valid probabilities and risk bands, and that missing columns in an uploaded file are reported. The application was also tested by hand with typical, high-risk and inconsistent inputs and with valid and invalid CSV files.

5.6 Source Code Repository

The complete project, including code, data, the trained model, charts, tests, this report and the presentation, is published on GitHub at:

`https://github.com/varunmundhada/employee-attrition-prediction`

The README gives one-command set-up and run instructions (Appendix B).

6. RESULTS AND DISCUSSION

6.1 Model Comparison

Table 6.1 compares the four tuned models. CV AUC is the mean over the five folds and was used for model selection; OOF AUC is computed on the out-of-fold predictions used for the threshold. All other columns are on the untouched test set of 221 employees, at each model's tuned threshold.

Table 6.1: Model comparison (test-set metrics at the tuned threshold; best model in bold)

Model	**CV AUC**	**OOF AUC**	**Test AUC**	**Acc.**	**Prec.**	**Recall**	**F1**	**Thr.**
Logistic Regression	**0.839**	**0.843**	**0.813**	**0.846**	**0.526**	**0.556**	**0.540**	**0.68**
Decision Tree	0.679	0.737	0.807	0.805	0.431	0.611	0.506	0.67
Random Forest	0.804	0.807	0.839	0.855	0.556	0.556	0.556	0.38
Gradient-Boosted Trees	0.774	0.772	0.827	0.833	0.488	0.556	0.519	0.54

Table 6.2: Selected hyper-parameters and training time (grid search with 5-fold CV)

Model	**Best parameters**	**Time (s)**
Logistic Regression	regParam = 0.01, elasticNetParam = 0.5	423
Decision Tree	maxDepth = 7, minInstancesPerNode = 20	92
Random Forest	numTrees = 200, maxDepth = 8	158
Gradient-Boosted Trees	maxIter = 100, maxDepth = 3	1546

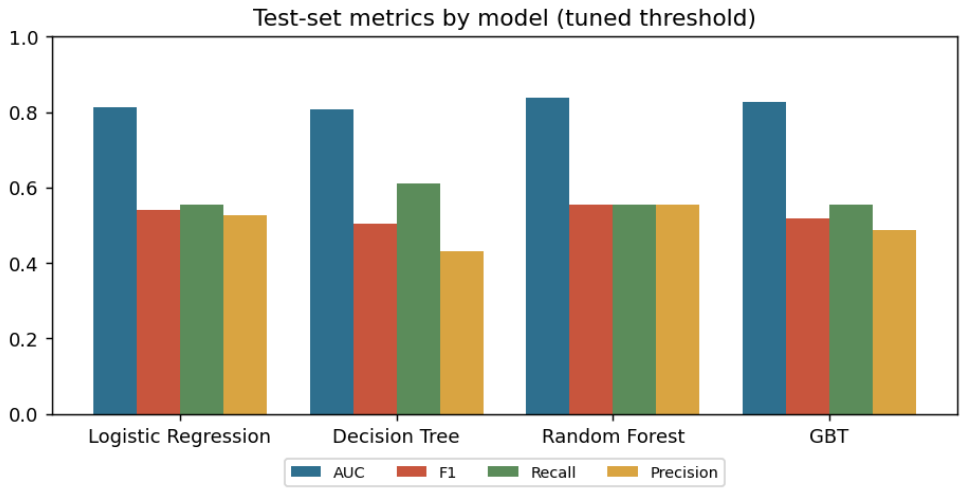
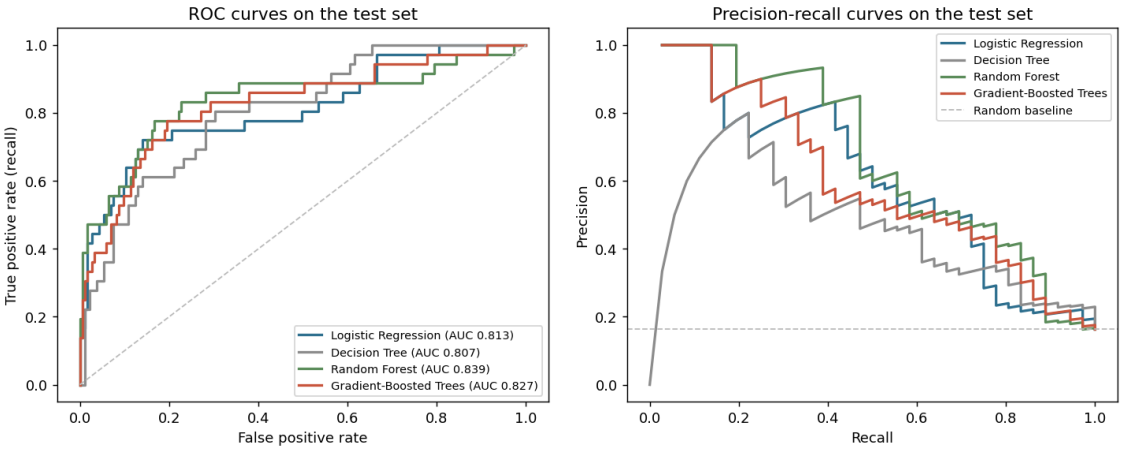


Figure 6.1: Test-set metrics by model

Logistic Regression was selected, with a cross-validated AUC of 0.839 and a test AUC of **0.813**. On the test set it reached an accuracy of 84.6%, a precision of 52.6%, a recall of 55.6% and an F1-score of 54.0% at a threshold of 0.68. The ROC and precision–recall curves (Figure 6.2) show that every model ranks employees far better than chance. On this small, mostly linear dataset the regularised logistic regression matches or beats the tree ensembles in cross-validation, in line with Zhao et al. [5], and it has the added advantage that its coefficients can be read directly. The single decision tree is clearly the weakest, as expected.

The test AUC (0.813) is lower than the cross-validated AUC (0.839). With only 36 leavers in the test set, a handful of employees can move the AUC by several points: during development, runs with different random splits gave test AUCs anywhere between 0.79 and 0.87 for the same models. The cross-validated figure, averaged over five folds of 1249 employees, is the more stable estimate of performance on new data, and the test figure is a conservative check that the model was not over-tuned.



(a) ROC curves

(b) Precision–recall curves

Figure 6.2: ROC and precision–recall curves on the test set

6.2 Effect of Threshold Tuning

Table 6.3 compares each model at the default threshold of 0.5 with the threshold tuned on the out-of-fold predictions. Because class weights already rebalance the training, all thresholds stay near the middle of the range, and tuning changes F1 only slightly. What tuning really changes is the balance between precision and recall. For the selected model, the F1-optimal threshold of 0.68 finds 55.6% of the leavers with a precision of 52.6%, while the default of 0.5 finds 75% of them but with a precision of only 42%. The threshold is therefore a business choice: an HR team that can hold many stay interviews should use a lower threshold to catch more leavers, and a team with limited capacity a higher one. The application shows the threshold on the risk gauge, and the Medium risk band covers employees just below it.

Table 6.3: Default (0.5) versus tuned threshold on the test set

	Threshold 0.5			**Tuned threshold**		
Model	**Prec.**	**Recall**	**F1**	**Thr.**	**Recall**	**F1**
Logistic Regression	0.415	0.750	0.535	0.68	0.556	0.540
Decision Tree	0.348	0.639	0.451	0.67	0.611	0.506
Random Forest	0.824	0.389	0.528	0.38	0.556	0.556
Gradient-Boosted Trees	0.511	0.639	0.568	0.54	0.556	0.519

6.3 Confusion Matrix

Of the 36 employees in the test set who actually left, the selected model correctly flagged 20 and missed 16. Of the employees who stayed, it raised 18 false alarms and correctly cleared 167 (Figure 6.3). For HR, a list of flagged employees in which a substantial share are true leavers is a far better use of retention effort than reviewing everyone.

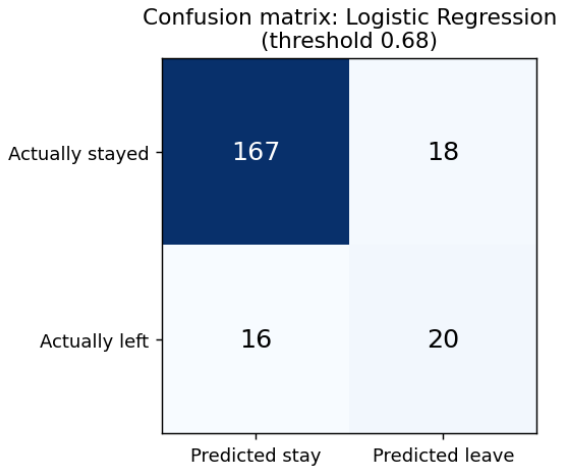


Figure 6.3: Confusion matrix of the selected model on the test set

6.4 Feature Importance

Figure 6.4 and Table 6.4 list the most influential features of the selected model; the top feature is **YearsSinceLastPromotion**. Because the selected model is a logistic regression on standardised features, each bar is a coefficient. A red bar (positive) raises the odds of leaving, and a blue bar (negative) lowers them, and the bar lengths can be compared directly. The ranking agrees with the exploratory analysis and with the literature [6]. A long time since the last promotion, working overtime, frequent business travel, many previous employers, a long commute and being single raise the risk. A high overall satisfaction index, a long relationship with the current manager, more years in the current role, higher environment satisfaction and more training lower it. Overtime appears twice (`Yes` and `No`) because each category has its own one-hot slot; the two coefficients are mirror images. The engineered `SatisfactionIndex` is the second-strongest feature, which supports the feature-engineering step. Most of these factors (promotion, overtime, travel, satisfaction, manager relationship, training) are *actionable*, so HR can act on them directly.



Figure 6.4: Top 15 features of the selected model

Table 6.4: Top 10 features of the selected model

Rank	Feature	Importance
1	YearsSinceLastPromotion	0.4460
2	SatisfactionIndex	0.3968
3	OverTime = Yes	0.3445
4	OverTime = No	0.3445
5	BusinessTravel = Travel_Frequently	0.2867
6	YearsWithCurrManager	0.2738
7	NumCompaniesWorked	0.2654
8	YearsInCurrentRole	0.2571
9	DistanceFromHome	0.2541
10	JobRole = Research Director	0.2535

6.5 Scalability on Larger Data

To show that the solution is a big-data pipeline and not only a small-data model, the dataset was enlarged synthetically: each record was copied up to 500 times, with $\pm 5\%$ random noise on nine numeric columns, to reach about 735,000 records. The same Spark pipeline

was trained on each size with fixed parameters (Table 6.5, Figure 6.5). The code did not change at any size. Training time grows much more slowly than the data. Going from 14,700 to 735,000 rows (50 times more data), logistic-regression training went from about 12 to 27 seconds and random-forest training from about 9 to 54 seconds on one laptop, because Spark spreads the work over all processor cores. On a cluster the same code would spread across machines. The first, smallest run is slower than the second because it includes Spark's start-up and JVM warm-up.

Table 6.5: Training and scoring time as the data grows (Spark local mode, one laptop)

Rows	**Model**	**Fit (s)**	**Score (s)**	**AUC**
1,470	Logistic Regression	22.1	3.2	0.800
1,470	Random Forest	11.0	1.4	0.806
14,700	Logistic Regression	11.9	1.4	0.863
14,700	Random Forest	8.5	1.7	0.995
73,500	Logistic Regression	10.8	1.5	0.874
73,500	Random Forest	13.1	1.4	0.997
294,000	Logistic Regression	16.2	2.1	0.871
294,000	Random Forest	27.0	2.7	0.995
735,000	Logistic Regression	27.2	3.8	0.873
735,000	Random Forest	53.7	3.4	0.995

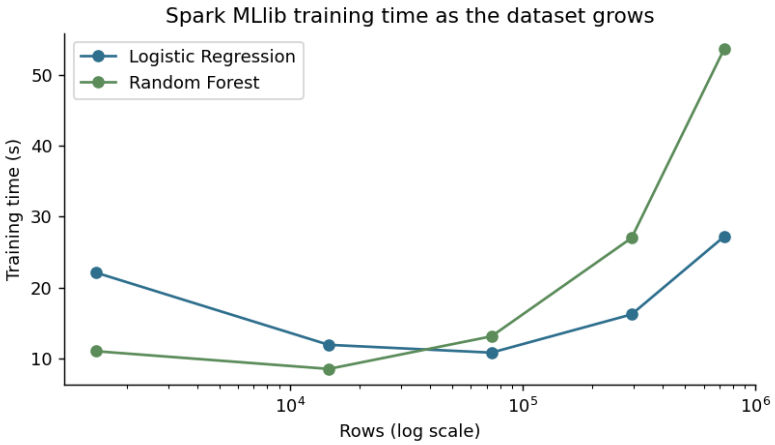


Figure 6.5: Training time versus data size

The AUC on the enlarged data is higher than on the original data because copies of the same employee can fall in both the training and the test part, so these AUC values

measure the pipeline's behaviour at scale, not its accuracy. The accuracy results are those in Table 6.1.

6.6 Business Insights and Recommendations

Table 6.6: Findings and recommended HR actions

Finding	**Recommended action**
Overtime workers leave at 30.5% vs 10.4%	Monitor overtime hours; rebalance workload or add headcount in teams with sustained overtime
Highest risk in the first two years (34.9%)	Structured onboarding, buddy programme and 30/60/90-day check-ins
Low income and job level 1 roles at highest risk	Benchmark entry-level pay; publish clear progression paths from level 1
No stock options: 24.4% attrition	Extend stock options or retention bonuses to high performers at level 0
Frequent travel: 24.9% attrition	Rotate travel duties; offer travel allowances or remote alternatives
Low satisfaction and poor work-life balance	Regular pulse surveys; stay interviews for low scorers; flexible hours
Sales Representatives at 39.8%	Targeted retention plan for the sales-representative role (incentives, coaching)

6.7 Limitations

- The dataset is synthetic, created by IBM, and has only 1,470 records, so the test set is small and the metrics carry some uncertainty.
- Each record is a snapshot; the model does not see how an employee's situation changed over time.
- The what-if explanation changes one factor at a time and does not capture interactions between factors.
- The model learns from past decisions and could carry their biases. Protected attributes such as gender and age must be reviewed before use on real data.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project built a complete, working employee attrition prediction system on Apache Spark. Exploratory analysis with Spark SQL showed that attrition concentrates among young, early-career, lower-paid employees, especially those working overtime, travelling frequently, without stock options and with low satisfaction. A Spark MLlib pipeline with five engineered features was used to train and tune four classifiers by cross-validation, with class weights for the imbalanced target. Selecting the model and threshold on the development data alone kept the final test evaluation unbiased.

The selected model, Logistic Regression, achieved a test ROC-AUC of 0.813 and found 55.6% of the employees who left. The model is deployed in a Streamlit application that gives HR a dashboard, individual risk predictions with explanations and suggested actions, and batch scoring. A scalability test showed the pipeline running unchanged on about 735,000 records. All objectives set in Chapter 1 were met.

Table 7.1: Objectives versus achievement

Objective	**Achieved**
Analyse attrition factors with Spark	Spark SQL analysis of 12 factors and an MLlib correlation matrix (Chapter 3)
Reusable Spark MLlib pipeline	Six-stage pipeline with in-pipeline feature engineering (Section 4.4)
Train and compare four classifiers	LR, DT, RF and GBT tuned with 5-fold CV (Table 6.1)
Unbiased selection and evaluation	85/15 stratified split; CV and out-of-fold tuning; test used once
Explain predictions	What-if analysis with suggested HR actions (Section 4.9)
Working application and scalability	Streamlit application (Chapter 5); test up to about 735k rows (Section 6.5)

7.2 Future Scope

- **Real HR data and time series**: train on an organisation's own records, adding monthly snapshots so the model sees trends such as falling satisfaction or rising overtime.
- **More data sources**: attendance, leave, engagement surveys and performance reviews, joined in Spark, following the "big data to deep data" direction of Yahia et al. [7].
- **Cluster deployment and scheduling**: run the pipeline on a Spark cluster (for example Databricks or Amazon EMR) with monthly retraining and drift monitoring.
- **Richer explanations**: SHAP values to capture interactions between factors.
- **Fairness auditing**: check prediction rates across gender and age groups before any real use.
- **Survival analysis**: predict *when* an employee is likely to leave, not only whether.

REFERENCES

[1] D. G. Allen, P. C. Bryant, and J. M. Vardaman, "Retaining talent: Replacing misconceptions with evidence-based strategies," *Academy of Management Perspectives*, vol. 24, no. 2, pp. 48–64, 2010.

[2] V. V. Saradhi and G. K. Palshikar, "Employee churn prediction," *Expert Systems with Applications*, vol. 38, no. 3, pp. 1999–2006, 2011.

[3] D. Alao and A. B. Adeyemo, "Analyzing employee attrition using decision tree algorithms," *Computing, Information Systems, Development Informatics and Allied Research Journal*, vol. 4, no. 1, pp. 17–28, 2013.

[4] R. Punnoose and P. Ajit, "Prediction of employee turnover in organizations using machine learning algorithms," *International Journal of Advanced Research in Artificial Intelligence*, vol. 5, no. 9, pp. 22–26, 2016.

[5] Y. Zhao, M. K. Hryniewicki, F. Cheng, B. Fu, and X. Zhu, "Employee turnover prediction with machine learning: A reliable approach," in *Intelligent Systems and Applications (IntelliSys 2018)*, ser. Advances in Intelligent Systems and Computing, vol. 869. Springer, 2019, pp. 737–758.

[6] F. Fallucchi, M. Coladangelo, R. Giuliano, and E. W. De Luca, "Predicting employee attrition using machine learning techniques," *Computers*, vol. 9, no. 4, p. 86, 2020.

[7] N. B. Yahia, J. Hlel, and R. Colomo-Palacios, "From big data to deep data to support people analytics for employee attrition prediction," *IEEE Access*, vol. 9, pp. 60 447–60 458, 2021.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[10] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] IBM, "IBM HR analytics employee attrition & performance," Kaggle dataset, 2017, accessed: Oct. 22, 2026. [Online]. Available: https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

[14] The Apache Software Foundation, "Spark MLlib guide, apache spark 3.5," 2024, accessed: Oct. 22, 2026. [Online]. Available: https://spark.apache.org/docs/3.5.3/ml-guide.html

[15] Snowflake Inc., "Streamlit documentation," 2026, accessed: Oct. 22, 2026. [Online]. Available: https://docs.streamlit.io

A. APPLICATION SCREENSHOTS

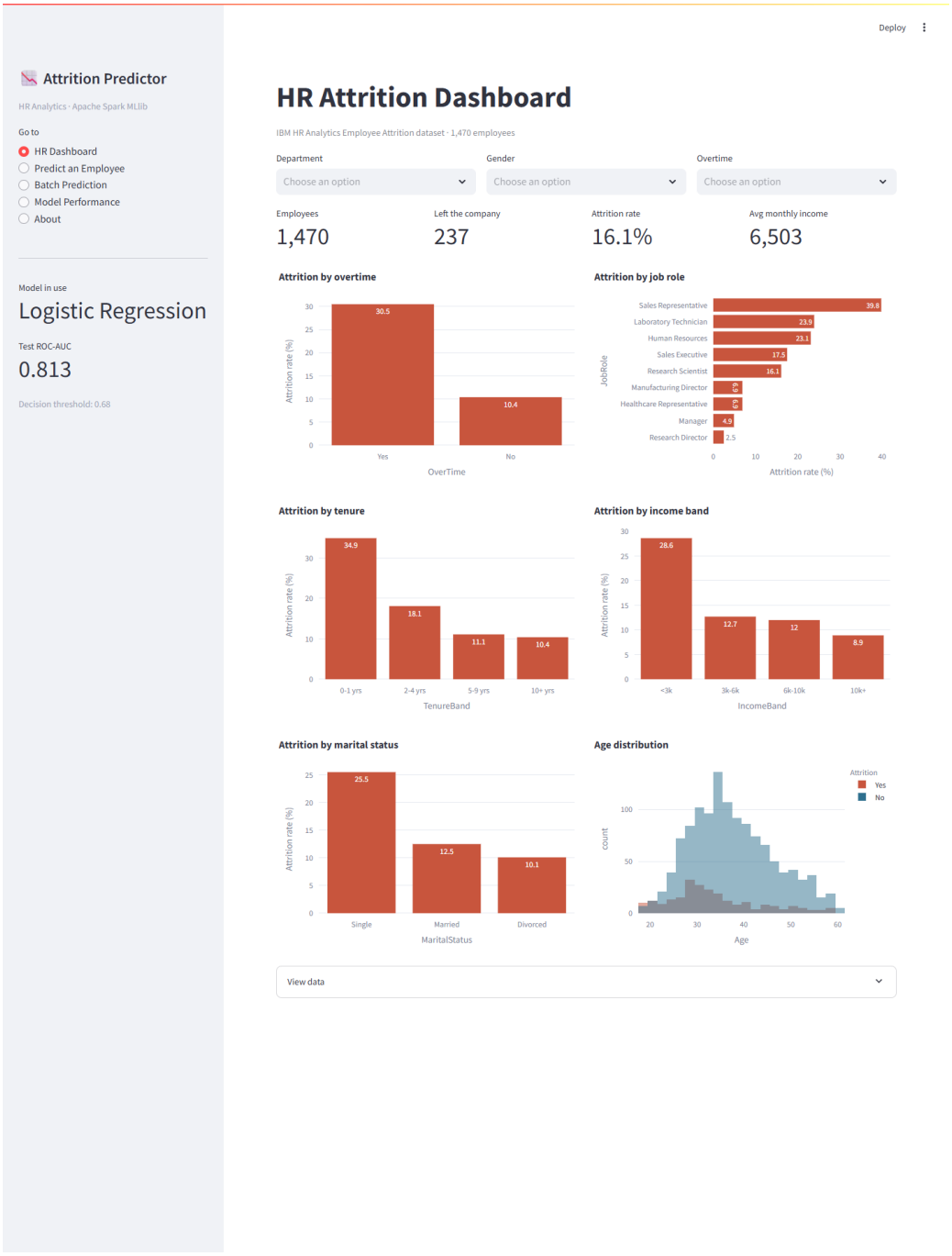


Figure A.1: HR Dashboard page

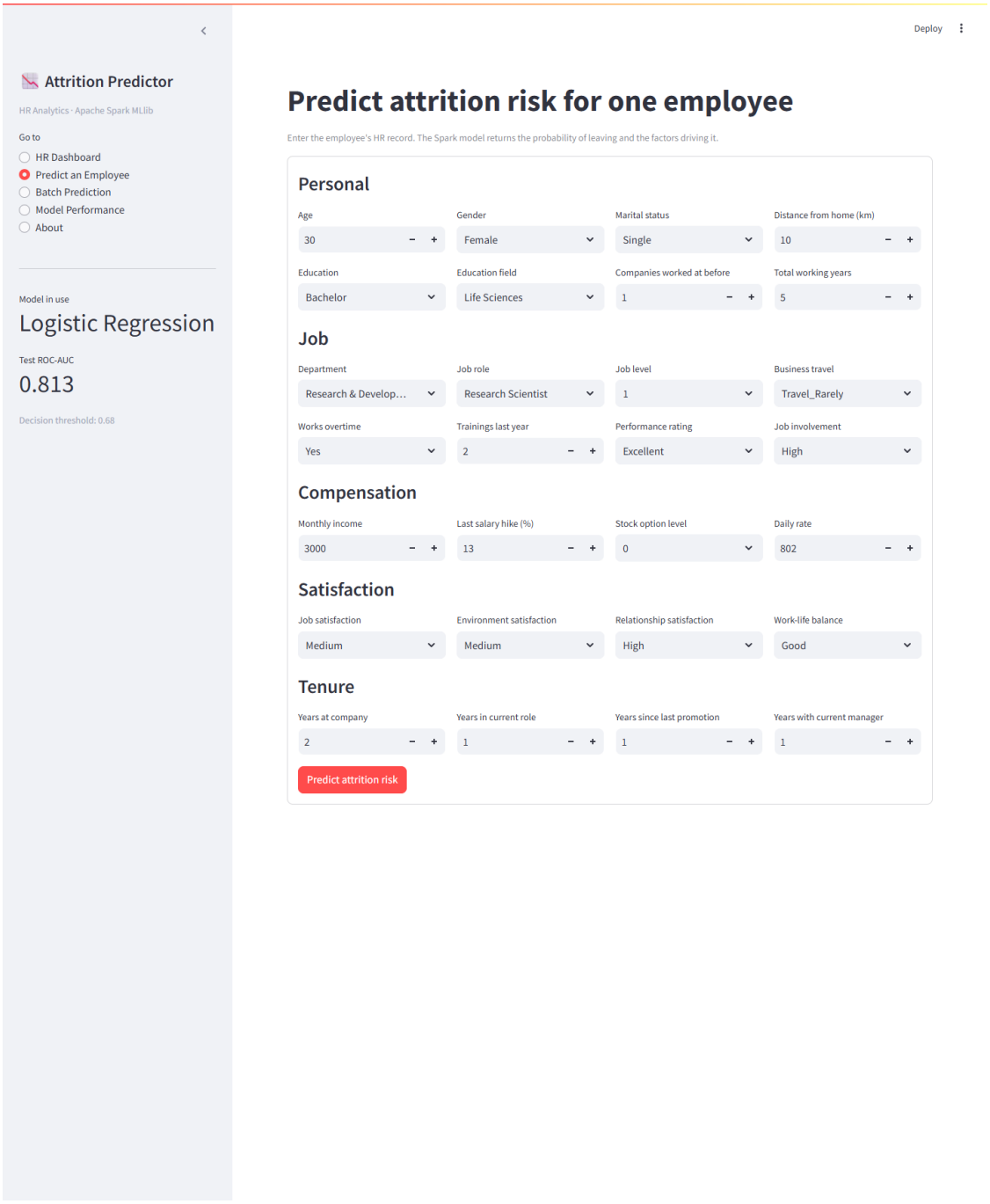


Figure A.2: Predict an Employee: input form

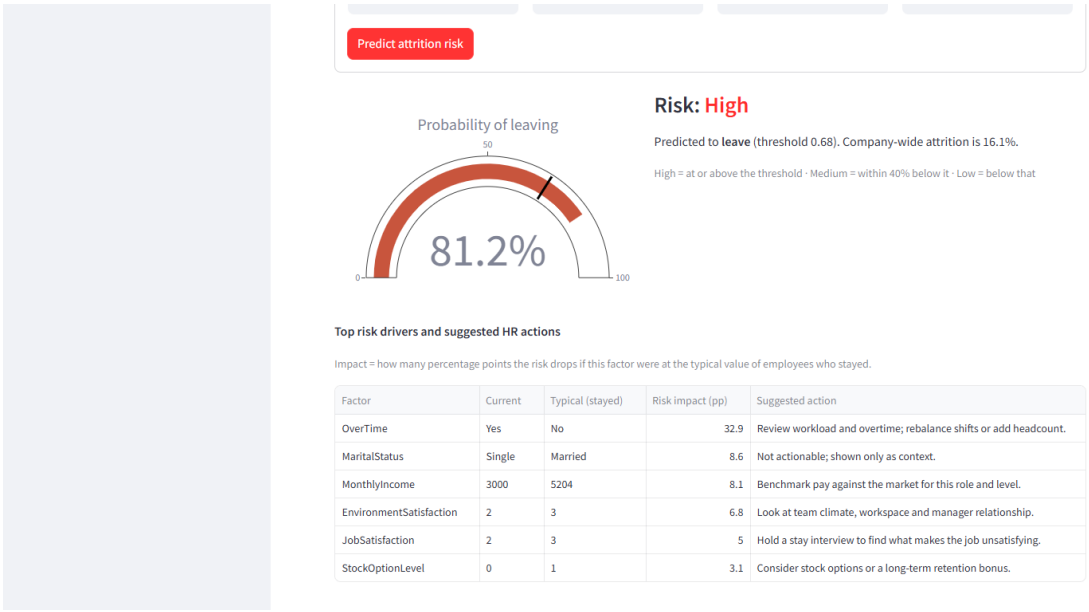
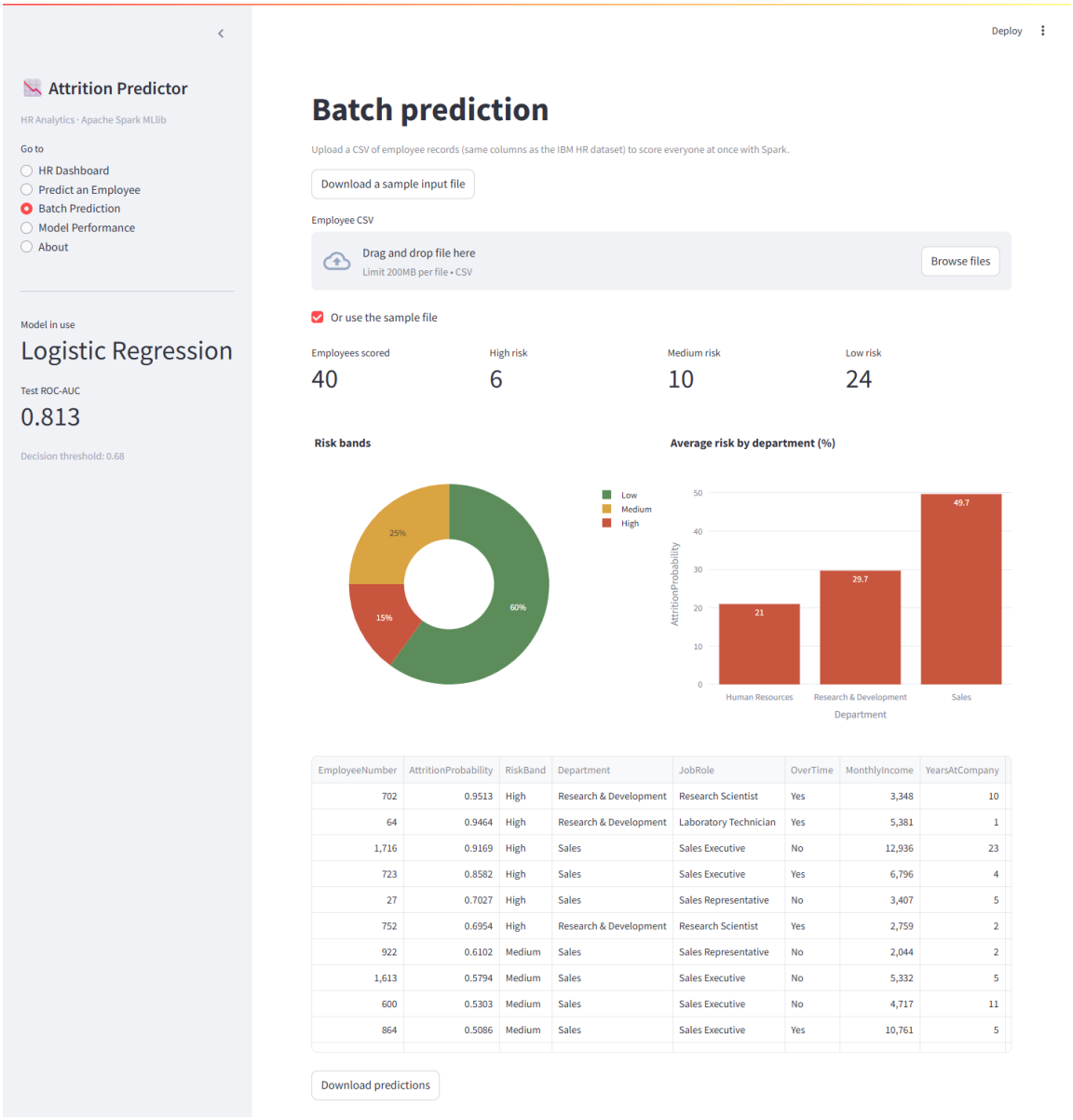


Figure A.3: Predict an Employee: risk gauge, drivers and suggested actions



EmployeeNumber	AttritionProbability	RiskBand	Department	JobRole	OverTime	MonthlyIncome	YearsAtCompany
702	0.9513	High	Research & Development	Research Scientist	Yes	3,348	10
64	0.9464	High	Research & Development	Laboratory Technician	Yes	5,381	1
1,716	0.9169	High	Sales	Sales Executive	No	12,936	23
723	0.8582	High	Sales	Sales Executive	Yes	6,796	4
27	0.7027	High	Sales	Sales Representative	No	3,407	5
752	0.6954	High	Research & Development	Research Scientist	Yes	2,759	2
922	0.6102	Medium	Sales	Sales Representative	No	2,044	2
1,613	0.5794	Medium	Sales	Sales Executive	No	5,332	5
600	0.5303	Medium	Sales	Sales Executive	No	4,717	11
864	0.5086	Medium	Sales	Sales Executive	Yes	10,761	5

Figure A.4: Batch Prediction page

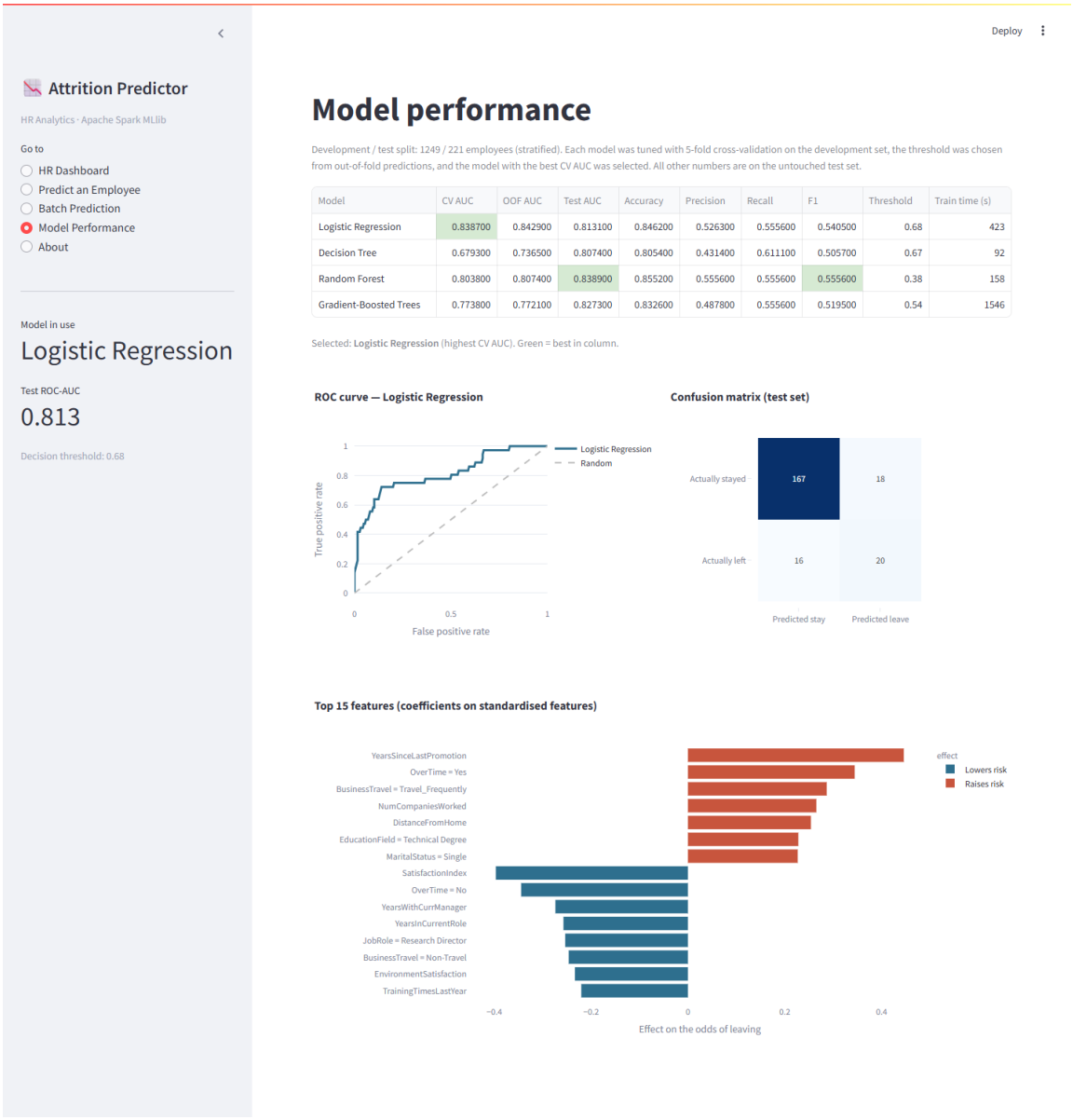


Figure A.5: Model Performance page

B. HOW TO RUN THE PROJECT

Prerequisites: Python 3.11, Java 17. On Windows, `winutils.exe` and `hadoop.dll` (Hadoop 3.3) in `%USERPROFILE%\hadoop\bin`, which Spark needs to save models.

```
git clone https://github.com/varunmundhada/employee-attrition-prediction.git
cd employee-attrition-prediction
python -m venv .venv
.venv\Scripts\activate # Linux / macOS: source .venv/bin/activate
pip install -r requirements.txt

python -m src.eda # analysis + charts
python -m src.train # train, compare, save best model
python -m pytest -q # tests
streamlit run app/streamlit_app.py # open http://localhost:8501

python -m src.predict data/sample_employees.csv out.csv # CLI scoring
python -m src.scale_benchmark # scalability test
```

On Windows the batch files `setup.bat`, `run_pipeline.bat` and `run_app.bat` run the same steps.

C. PLAGIARISM REPORT

Plagiarism Report

Turnitin similarity report for this document, to be inserted after the final draft is checked.

D. AI SIMILARITY REPORT

AI Similarity Report

AI similarity report for this document, to be inserted after the final draft is checked.